

AgentReliabilityLab

Interview Defense Pack — FAANG Standard Edition v2

Agentic Cyber Alert Triage · MITRE ATT&CK; v14 · NIST CSF 2.0 · CISA KEV

LangGraph · Hybrid RAG · LangSmith · RAGAS · FastAPI · Docker

Sidharth Kriplani · github.com/SidharthKriplani/agentreliabilitylab

40+ Q&A; Pairs · Honest Positioning · Failure Modes · Scalability Analysis

Claim Verification Labels

[IMPLEMENTED]	Code exists, runs, produces verified artifact
[SIMULATED]	Deliberately synthetic; simulation logic is real code
[PARTIALLY VERIFIED]	Code exists but verification relies on internal consistency
[FUTURE]	Not built — explicitly absent from the repo

Contents

- 1. Project Overview & Honest Positioning
- 2. Architecture Deep-Dive
- 3. The Agentic Loop — Why It Matters
- 4. Hybrid RAG Design + RAGAS Results
- 5. Deterministic Overrides
- 6. LangSmith Observability
- 7. Human-in-the-Loop (HITL)
- 8. Failure Modes Taxonomy
- 9. Scalability Architecture
- 10. Resume Claim Audit
- 11. Do Not Say This
- 12. Interview Q&A; — 40 Questions

Project Overview & Honest Positioning

What AgentReliabilityLab Does

AgentReliabilityLab is a 6-node LangGraph agent that triages raw security alerts against a hybrid RAG knowledge base covering MITRE ATT&CK v14, NIST CSF 2.0, and CISA KEV advisories. The agent accepts raw alert text in any format — EDR JSON, Syslog, IDS trigger, SIEM event — and produces: severity classification (CRITICAL/HIGH/MEDIUM/LOW/BENIGN), MITRE technique mapping, recommended NIST controls, IOC extraction, and a tamper-evident audit log, within 15–40 seconds on local hardware.

What Makes It an Agent (Not a Pipeline)

The conditional edge from analyze → retrieve. After the first retrieval + analysis pass, if the LLM's self-reported confidence is below threshold, the graph loops back with a refined query incorporating technique IDs and tactic names from the partial analysis. This feedback loop — bounded at 2 attempts — is encoded as a LangGraph conditional_edge (not application-level code), making it traceable in LangSmith, checkpointable for HITL, and testable in isolation.

Honest Positioning: What Is Real vs. Simulated

Be able to say every line below without hesitation. Interviewers at FAANG-level respect honesty far more than inflation. The right framing: 'production architecture, curated data.'

Claim	Status	Details
6-node LangGraph pipeline	[IMPLEMENTED]	intake→retrieve→analyze→confidence→route→audit in pipeline/graph.py
Conditional re-retrieval loop	[IMPLEMENTED]	_should_reretrieve() conditional edge; bounded at MAX_RETRIEVAL_ATTEMPTS=2
Hybrid BM25 + dense retrieval	[IMPLEMENTED]	rank-bm25 + ChromaDB/all-MiniLM-L6-v2 + cross-encoder reranker
MITRE ATT&CK; v14 knowledge base	[SIMULATED]	Curated excerpts of key techniques, NOT the full STIX/TAXII corpus (~900+ techniques)
NIST CSF 2.0 controls	[SIMULATED]	Selected control descriptions, not full 108 subcategories
CISA KEV advisories	[SIMULATED]	Sample advisories, not the live KEV feed (~1,000+ entries updated daily)
Confidence thresholds (0.65/0.80)	[SIMULATED]	Heuristic constants hand-tuned on test cases — NOT calibrated against labelled alerts
RAGAS scores (0.923/0.891/0.847/0.814)	[PARTIALLY VERIFIED]	Measured on N=5 synthetic evaluation fixtures, not production-scale
Real SOC alert stream	[FUTURE]	No CrowdStrike/SentinelOne/Splunk integration. Demo uses crafted test cases
CERT-In 6-hour reporting	[SIMULATED]	Audit log structure supports compliance; CERT-In API integration not built

FastAPI + HITL API	[IMPLEMENTED]	/triage, /human-review/override. Tested with mocked LLM in CI
LangSmith observability	[IMPLEMENTED]	LANGCHAIN_TRACING_V2=true; LangSmith run_id stored in every audit record
Docker deployment	[IMPLEMENTED]	Dockerfile + docker-compose.yml; runs with LM Studio

Architecture Deep-Dive

6-Node Pipeline [IMPLEMENTED]

Node	Input	Output	Key Design Choice
intake	raw_alert (str)	alert_id, source, metadata	Heuristic source detection; deterministic benign pre-check
retrieve	alert + metadata + query	retrieved_chunks, attempt++	Hybrid BM25 + dense; enriched query on re-retrieval
analyze	alert text + chunks	TriageAnalysis Pydantic model	3-pass JSON recovery; SHA-256 cache; benign fast-path skips LLM
[loop edge]	requires_more_context AND attempts < max	→ retrieve OR → confidence	LangGraph conditional_edge — the agentic feedback loop
confidence	analysis dict	confidence_score, confidence_band (H/M/L)	+0.05/technique ID; +0.05/NIST control; -0.10/UNKNOWN severity
route	analysis + confidence	triage_decision, reason_codes, human_review_required	Technique hard overrides; LOW confidence upgrades LOW→MEDIUM
audit	full AlertState	audit_id, JSONL record	Chunk IDs only; LangSmith run_id pointer; token economics logged

Conditional Edge Code [IMPLEMENTED]

```
def _should_reretrieve(state): if state.get('error'): return 'confidence' # propagate error
analysis = state.get('analysis', {}) attempts = state.get('retrieval_attempts', 0)
if analysis.get('requires_more_context') and attempts < MAX_RETRIEVAL_ATTEMPTS: return 'retrieve' # agentic loop back
return 'confidence' # proceed
```

The Agentic Loop — Why It Matters

Without re-retrieval, an alert containing both a generic process name and a known CVE might retrieve only process-related chunks on the first pass, missing the CVE-specific guidance that would change the routing decision. The loop catches these cases by using the partial analysis (technique_ids, tactics) as a refined query on the second pass.

- **Pass 1:** Query built from raw alert text + extracted metadata. Top-8 chunks via hybrid BM25 + dense.
- **Analysis 1:** LLM produces TriageAnalysis with requires_more_context flag, technique_ids, severity.
- **Loop check:** requires_more_context=True AND attempts < 2? → loop back to retrieve.
- **Pass 2:** Query enriched with technique_ids and tactics from Analysis 1 — much more specific.
- **Analysis 2:** LLM re-analyzes with richer context. Sets requires_more_context=False. Proceeds.

Why Bounded at 2 Attempts?

Unbounded loops are a reliability anti-pattern. After 2 passes, additional context diminishes (the knowledge base only has what it has) while latency grows linearly. MAX_RETRIEVAL_ATTEMPTS is a config key — tunable per deployment. The bound is enforced by a state counter check BEFORE the conditional edge, not by the LLM, which cannot be trusted to self-terminate.

Hybrid RAG Design + RAGAS Results

Layer	Tool	Why	Status
Dense	ChromaDB + all-MiniLM-L6-v2	Semantic similarity — catches paraphrased attack descriptions	[IMPLEMENTED]
Sparse	BM25 (rank-bm25)	Exact keyword — CVE IDs, technique IDs, tool names (Mimikatz)	[IMPLEMENTED]
Rerank	cross-encoder/ms-marco-MiniLM-L-6-v2	Joint query-document scoring — lifts precision without hurting recall	[IMPLEMENTED]
Dedup	chunk_id keyed dict	Same chunk from both retrievers → keep highest score, no duplicate	[IMPLEMENTED]

RAGAS Results [PARTIALLY VERIFIED — N=5 synthetic fixtures]

Metric	Score	What It Measures	Interpretation
context_recall	0.923	Were right chunks retrieved?	BM25 captures exact IDs; dense captures paraphrases
answer_relevancy	0.891	Does answer address the alert?	Output is alert-specific, not generic security advice
faithfulness	0.847	Does answer contradict chunks?	Mostly grounded; some off-chunk model priors in edge cases
context_precision	0.814	Are retrieved chunks useful?	Cross-encoder reduces off-topic NIST subcategories

Retrieval Ablation [PARTIALLY VERIFIED]

Strategy	Recall	Precision	Finding
BM25 only	0.741	0.823	Misses semantic paraphrases of attack patterns
Dense only	0.868	0.779	Misses exact technique IDs; lower precision
Hybrid (no reranker)	0.901	0.791	Better recall but off-topic chunks slip through
Hybrid + reranker	0.923	0.814	Best on both axes — each component earns its place

Deterministic Overrides

Principle: never use a probabilistic model for what you can compute with certainty. This mirrors LendFlow's deterministic FOIR recomputation. Two categories:

Category A — Technique Hard Overrides (route node) [IMPLEMENTED]

Technique	Override	Rationale
T1486 — Data Encrypted for Impact	→ CRITICAL	Ransomware payload in progress
T1490 — Inhibit System Recovery	→ CRITICAL	Shadow copy deletion — ransomware precursor
T1485 — Data Destruction	→ CRITICAL	Irreversible damage in progress
T1561 — Disk Wipe	→ CRITICAL	Destructive attack
T1055 — Process Injection	→ HIGH	Active code injection
T1003 — OS Credential Dumping	→ HIGH	LSASS/Mimikatz activity
T1078 — Valid Accounts	→ HIGH	Privilege escalation
T1021 — Remote Services	→ HIGH	Lateral movement
T1071 — C2 Protocol	→ HIGH	Active C2 channel
T1190 — Exploit Public-Facing App	→ HIGH	Active exploitation attempt

Category B — Deterministic Benign Fast-Path [IMPLEMENTED]

Known-safe phrase combinations detected at intake node: ('scheduled' + 'deploy'), ('nessus' + 'scan'), ('ci/cd' + 'deploy'), ('vulnerability scanner'), ('patch tuesday'). Result: alert_metadata['deterministic_benign']=True → BENIGN at 0.95 confidence with 0 LLM tokens consumed. Observed latency: 0.4 seconds.

LangSmith Observability

[IMPLEMENTED] `LANGCHAIN_TRACING_V2=true` enables full tracing. Every pipeline run produces a LangSmith trace showing per-node latency, token cost, retrieval queries, full prompt/response chain, and re-retrieval loop visibility. The `langsmith_run_id` is stored in every audit log record.

- Per-node latency: which node is the bottleneck? (Typically analyze — the LLM call)
- Token usage: how many tokens per call? Did re-retrieval double the cost?
- Full prompt + raw response: exact prompt sent including retrieved chunks
- Re-retrieval trace: if loop triggered, both passes appear as separate spans with queries
- `langsmith_run_id`: every audit record links back to its full LangSmith trace

Design is opt-in: `LANGCHAIN_TRACING_V2=false` (default) — pipeline runs with zero external dependencies. One env variable flips full observability on.

Human-in-the-Loop (HITL)

Graph Variant	Compilation	Used By	Behavior
build_graph()	interrupt_before=['audit']	FastAPI /triage endpoint	Pauses before audit for CRITICAL/HIGH; analyst can override
build_graph_no_interrupt())	No interrupts	demo.py, run_eval.py, tests, CI	Audit always runs; audit_id always populated

Why Two Graph Variants? (The Production Bug This Fixes)

interrupt_before=['audit'] means the audit node never runs in batch contexts — demo.py, run_eval.py, tests — because those don't use the HITL resume flow. Every alert would show audit_id='—'. Fix: compile a second graph without interrupt. run_pipeline() always uses graph_no_interrupt. This is the kind of subtle production semantics that shows you've actually run LangGraph beyond the tutorial.

Resume Flow [IMPLEMENTED]

POST /human-review/override with {thread_id, alert_id, override_decision, analyst_notes}. Calls graph.update_state() to inject override into checkpoint, then graph.stream() to resume. Audit log captures LLM's original decision AND analyst override with reason field.

Failure Modes Taxonomy

A senior engineer will ask: 'What breaks?' You need a taxonomy, not an excuse. Knowing your own failure modes is a sign of engineering maturity.

Failure Mode	Trigger	Mitigation	Status
LLM JSON parse failure	Local 4B model leaks markdown fences	3-pass recovery → MAX_RETRIES → NEEDS_REVIEW sentinel	[IMPLEMENTED]
Retrieval miss — novel technique	Zero-day not in curated corpus	LLM uses parametric knowledge; confidence drops; human review fires. Fix: weekly STIX/TAXII ingestion [FUTURE]	[SIMULATED]
Confidence threshold miscalibration	Heuristic +0.05/technique may over-promote noisy detections	Technique hard overrides provide floor. Production: isotonic calibration on labelled alert history	[FUTURE]
Unbounded loop	requires_more_context stuck at True	MAX_RETRIEVAL_ATTEMPTS=2 enforced by state counter check BEFORE edge — not by LLM	[IMPLEMENTED]
Silent LLM timeout	LM Studio hangs under load	async astream() with timeout. LangSmith shows hanging node. Production: circuit breaker around OpenAI calls	[PARTIALLY VERIFIED]
Context window overflow	8 chunks × 400 chars exceeds model context	token_log tracks usage per node; LangSmith shows spike. Fix: reduce top-K or chunk compression	[SIMULATED]
False BENIGN fast-path	Attacker names C2 binary 'scheduled_deploy.exe'	Fast-path requires phrase combinations, not single keywords. Configurable, can be disabled	[IMPLEMENTED]
Multi-stage attack context loss	Alert 3 needs context from Alert 1	Current design: stateless per alert. Production: session context store per source entity [FUTURE]	[FUTURE]
Audit log corruption	Process kill mid-write → partial JSONL record	Append-only; partial records detectable. Production: S3 atomic multipart upload	[FUTURE]

SECTION 9

Scalability Architecture

Every item below is a config change or infrastructure swap — zero pipeline code rewrites required. The architecture was designed for this from the start.

Bottleneck	Current	Production Fix	Target
LLM throughput	LM Studio, 1 req/time	Swap LLM_BASE_URL → OpenAI/Azure OpenAI — 3 env vars, 0 code changes	10–50 req/s
Retrieval latency	ChromaDB in-process	ChromaDB server mode or Qdrant cluster. Retriever behind interface — 1 line	<10ms p99
Alert ingestion	POST /triage (sync)	Kafka topic → worker pool → graph.astream() async + dead letter queue	1000+ alerts/hr
HITL state	MemorySaver (lost on restart)	SqliteSaver or PostgresSaver — LangGraph built-in, swap checkpointinter param	Survives restart
Cache	In-memory dict (process-local)	Redis cluster — REDIS_URL env var, zero code changes	Cross-process dedup
Audit storage	Local JSONL files	S3 + DynamoDB or PostgreSQL. audit_node just needs a writer interface	Unlimited retention
Alert deduplication	SHA-256 cache (process-local)	Redis SETNX on alert hash — atomic dedup across workers	No double-processing
Multi-tenant SOC	Single tenant only	Tenant ID in AlertState + per-tenant KB namespacing in Qdrant	Isolated per SOC
Eval CI	pytest + mock LLM	eval-on-PR via GitHub Actions — already wired in .github/workflows/eval.yml	[IMPLEMENTED]

1000 Alerts/Hour — Worked Calculation

1000 alerts/hr = 0.28 alerts/sec. Current latency: 15–40s (LM Studio). With GPT-4o-mini at ~2s/analyze: total pipeline ~4s. 1 async worker: 3600/4 = 900/hr. 2 workers: 1800/hr — comfortably above target. True bottleneck: LLM inference. Cross-encoder reranker is second bottleneck at ~2-4s — could be replaced with lighter bi-encoder.

Resume Claim Audit

Resume Claim	Evidence	Status	Guidance
'6-node LangGraph agent'	pipeline/graph.py — build_graph()	[IMPLEMENTED]	Safe to say
'Conditional re-retrieval loop'	_should_reretrieve() + conditional_edges	[IMPLEMENTED]	Safe to say
'Hybrid RAG — BM25 + ChromaDB + reranker'	pipeline/nodes/retrieve.py	[IMPLEMENTED]	Safe to say
'RAGAS evaluation framework'	eval/ragas_eval.py; scores in README	[PARTIALLY VERIFIED]	Add: 'N=5 synthetic fixtures'
'MITRE ATT&CK; v14 knowledge base'	threat_docs/; curated excerpts	[SIMULATED]	Add: 'curated excerpts, not full corpus'
'LangSmith observability'	LANGCHAIN_TRACING_V2; run_id in audit	[IMPLEMENTED]	Safe to say
'HITL interrupt + resume'	interrupt_before=['audit']; /human-review/override	[IMPLEMENTED]	Safe to say
'Deterministic technique overrides'	pipeline/nodes/route.py — _technique_override()	[IMPLEMENTED]	Safe to say
'FastAPI serving'	api/ directory; /triage endpoint	[IMPLEMENTED]	Safe to say
'Real-time SOC alert stream'	Not built	[FUTURE]	Do NOT say this
'Production-deployed system'	Not deployed	[FUTURE]	Do NOT say this

Do Not Say This

Do NOT Say	Say Instead
'I built a production SOC triage system'	'Production-architecture agent demonstrating SOC triage — demo-grade system'
'My MITRE corpus covers all ATT&CK; techniques'	'Curated excerpts of MITRE ATT&CK; v14; production would ingest the full STIX/TAXII feed'
'Confidence threshold of 0.65 is calibrated'	'Heuristic threshold — production calibration requires isotonic regression on labelled alert history'
'RAGAS shows 0.923 context recall'	'RAGAS on 5 synthetic test cases shows 0.923 — directionally valid, not production-scale'
'I integrated with CrowdStrike/Splunk'	'Designed to accept any alert format; tested with synthetic EDR/SIEM/IDS alert structures'
'The agent handles multi-stage attacks'	'Current design is per-alert stateless; multi-stage context is a known gap [FUTURE]'
'Real-time sub-second triage'	'Async Tier-1 triage; with OpenAI GPT-4o-mini latency reduces to ~3-5s per alert'

Interview Q&A; — 40 Questions

First-principles mentor tone: WHY, not just WHAT. Lead every answer with the reason the design decision exists.

Q1. What is AgentReliabilityLab and what problem does it solve?

It's a LangGraph agent that triages raw security alerts against a hybrid RAG knowledge base of MITRE ATT&CK, NIST CSF 2.0, and CISA advisories. The problem: SOC analysts manually read thousands of alerts per day to find the handful that are genuine threats. ARL automates Tier-1 triage — classifying severity, mapping to MITRE techniques, recommending controls, and flagging for human review. The core thesis: an agent needs explicit termination conditions; ARL makes those rules deterministic and auditable.

Q2. What makes this an agent vs. a pipeline?

The conditional edge from analyze → retrieve. If the LLM's self-reported confidence is below threshold after the first retrieval pass, the graph loops back with a refined query built from the partial analysis. A pipeline is static and linear. An agent reasons about whether it has enough information to commit to a decision. That feedback loop — encoded as a LangGraph `conditional_edge`, not application-level code — is the agent.

Q3. Why LangGraph instead of a simple Python loop?

Three reasons. (1) Stateful checkpointing — the graph can be paused mid-run for HITL and resumed after an analyst override; a Python loop can't. (2) Conditional edges as first-class graph topology — the re-retrieval loop is declared in the graph, not buried in application code. (3) LangSmith integration — every node is traced with per-call latency and token cost. A Python while loop gives you none of that.

Q4. Explain the hybrid RAG retrieval strategy.

BM25 handles exact keyword matches — CVE IDs, MITRE technique IDs, tool names like Mimikatz. ChromaDB with sentence-transformers handles semantic similarity — paraphrased attack descriptions. The cross-encoder reranker rescores the merged candidate set end-to-end: unlike bi-encoder retrieval (scores query and document independently), the cross-encoder scores each pair jointly, producing higher-quality top-K selection. Same BM25 + dense + reranker stack as LendFlow — proven to outperform either method alone.

Q5. Why use BM25 specifically for security data?

Security documents contain exact identifiers that embedding models struggle with: 'T1059.001', 'CVE-2021-44228', 'vssadmin.exe delete shadows /all /quiet'. These are low-frequency tokens. BM25 is exact-match by design. When an alert contains 'CVE-2021-44228', BM25 ranks the Log4Shell advisory first regardless of what embeddings say. Ablation confirms: BM25-only achieves 0.823 precision vs dense-only 0.779 on security keyword-heavy queries.

Q6. How does the confidence scoring work — and is it calibrated?

[HONEST] The confidence scoring is heuristic, not calibrated. The LLM self-reports a float (0-1). The confidence node applies: +0.05/technique ID (max +0.15), +0.05/NIST control (max +0.10), -0.10/UNKNOWN severity. These constants are hand-tuned, not derived from calibration against labelled alert history. Production calibration would require isotonic regression mapping model scores to empirical false positive rates on real analyst-labelled data.

Q7. How does your confidence threshold relate to calibrated probabilities vs heuristic scores?

Fundamentally different things. A calibrated probability: if the model says 0.8 confidence, 80% of such alerts are truly HIGH severity — verified against ground truth labels. My threshold of 0.65 is heuristic: tuned by running the pipeline against synthetic test cases until HITL rate felt right for the demo. Calibration requires a labelled dataset of real alerts with analyst-verified severity, then isotonic regression or Platt scaling. My thresholds are starting points, not calibrated operating points.

Q8. Your MITRE corpus is curated excerpts — how would you expand to full ATT&CK; in production?

Full MITRE ATT&CK; has ~900+ techniques via the STIX/TAXII API at attack.mitre.org/taxii2/. Ingestion pipeline: (1) pull the full STIX bundle via TAXII 2.1 API, (2) parse each technique object (description, detection guidance, data sources), (3) chunk at 400 chars / 80-char overlap and index to Qdrant, (4) schedule weekly refresh via GitHub Actions cron job. The retriever interface is already abstracted — swapping the knowledge base requires no graph code changes. Key challenge: hierarchical chunking to preserve sub-technique relationships.

Q9. What's the latency of the full pipeline? Where's the bottleneck?

Local hardware, LM Studio Gemma 3 4B: 15-40 seconds end-to-end. From LangSmith traces: intake 0.1s, retrieve 2-4s (cross-encoder dominated), analyze 12-30s (LLM inference dominated), confidence 0.01s, route 0.01s, audit 0.1s. Bottleneck: unambiguously the LLM. With GPT-4o-mini: analyze drops to ~1-2s, total ~3-5s. Re-retrieval loop roughly doubles latency for the second LLM call.

Q10. How do you handle multi-stage attacks where context from alert 1 is needed for alert 3?

[HONEST] The current design does not handle this. Each alert is processed statelessly — the AlertState contains only the current alert's context, no historical alert memory. A multi-stage attack (T1566 phishing → T1059 execution → T1021 lateral movement) produces three independent triage decisions without connecting the dots. Production fix: session context store keyed by source IP/hostname/user entity; for each new alert, retrieve last N summaries for that entity and prepend to retrieval query. Known gap in current architecture.

Q11. What's CERT-In's 6-hour reporting mandate and how does your audit trail support it?

CERT-In (Computer Emergency Response Team India) mandates critical infrastructure operators report cyber incidents within 6 hours under the CERT-In Directions 2022. The audit log produces a tamper-evident JSONL record with: timestamp, triage_decision, technique_ids, severity_rationale, recommended_controls, human_review_required, human_override, langsmith_run_id. This provides structured evidence for incident reporting. The append-only JSONL format (no updates, no deletes) provides tamper evidence. [SIMULATED] CERT-In API integration for actual submission is not built.

Q12. Why are some techniques hard-coded as CRITICAL overrides?

T1486 (ransomware encryption) and T1490 (shadow copy deletion) are unambiguous ransomware precursors. Waiting for the LLM to correctly classify them introduces reliability risk — a 4B model might assign HIGH instead of CRITICAL. A 0.65-confidence detection of T1486 is still a ransomware incident. Tying CRITICAL escalation to confidence would mean a noisy shadow-deletion alert gets downgraded. Safety-critical paths must be deterministic.

Q13. What's the deterministic benign fast-path?

Known-safe phrase combinations at intake: ('scheduled' + 'deploy'), ('nessus' + 'scan'), ('ci/cd' + 'deploy'). Sets `alert_metadata['deterministic_benign']=True`. The `analyze` node returns pre-built BENIGN TriageAnalysis at 0.95 confidence — 0 LLM tokens, 0.4s latency. Prevents the model from hallucinating threat techniques onto clearly benign CI/CD activity. Fast-path is configurable and can be disabled for high-security environments.

Q14. How does the HITL flow work?

Production graph compiled with `interrupt_before=['audit']`. For CRITICAL/HIGH alerts, graph pauses before the audit node. `POST /human-review/override` accepts analyst's decision. `graph.update_state()` injects override into LangGraph checkpoint, then `graph.stream()` resumes. Audit log captures both LLM's original decision AND analyst override with reason field — critical for SOC audit trails and CERT-In compliance.

Q15. Why two graph variants (graph vs. graph_no_interrupt)?

`interrupt_before=['audit']` means the audit node never runs in batch contexts — `demo.py`, `run_eval.py`, tests — because those don't use the HITL resume flow. Every alert would show `audit_id='—'`. Fix: compile a second graph without interrupt for non-HITL contexts. `run_pipeline()` always uses `graph_no_interrupt`. Subtle but real production bug that shows you've run LangGraph beyond the tutorial.

Q16. How does LangSmith fit in?

`LANGCHAIN_TRACING_V2=true` → LangGraph automatically traces every execution. Each node appears as a span with input state, output state, latency, token cost. `langsmith_run_id` stored in every audit record — audit entry links to full trace. Agentic systems fail in non-obvious ways: silent prompt formatting errors, reranker returning off-topic chunks, model returning malformed JSON on the third pass — all invisible without structured observability.

Q17. What does RAGAS measure that your custom eval doesn't?

Custom eval (`run_eval.py`) measures triage accuracy — did the final decision match ground truth? RAGAS measures retrieval quality independently: `faithfulness` (does the LLM contradict chunks?), `context_recall` (were right chunks retrieved?), `context_precision` (are chunks useful or noisy?). You can have 90% triage accuracy while retrieval is broken — the LLM compensates with parametric knowledge. RAGAS catches that failure mode.

Q18. How is this different from a rule-based SIEM like Splunk or QRadar?

SIEM/SOAR platforms are rule-based playbook executors — a human writes 'if alert.type == ransomware then isolate host'. They don't understand unstructured alert text, don't do RAG retrieval, don't map to MITRE techniques automatically. ARL uses an LLM + knowledge base to understand arbitrary alert formats and reason about threat intel. The tradeoff: SIEM is deterministic and trusted by compliance; ARL is flexible but probabilistic. Production: ARL for Tier-1 classification feeding a SOAR playbook for response actions.

Q19. What would break first if you pointed this at real CrowdStrike alert data?

[HONEST] Several things in order: (1) JSON schema mismatch — CrowdStrike EDR alerts have nested schema; intake node's heuristic source detection needs a CrowdStrike-specific parser. (2) Knowledge base gaps — real CrowdStrike alerts reference techniques the curated corpus doesn't cover. (3) Alert volume — CrowdStrike can generate thousands of events/minute; synchronous /triage endpoint falls over without a queue. (4) False BENIGN hits — real telemetry includes process names that might accidentally match phrase patterns. All fixable — integration engineering, not architecture flaws.

Q20. Walk me through a phishing alert from ingestion to output — every node.

Input: 'User clicked link. Process outlook.exe spawned powershell.exe with encoded command.'
INTAKE: source=EDR. Not benign fast-path. alert_id generated. RETRIEVE: BM25 matches 'powershell.exe encoded command' → T1059.001 chunks. Dense matches T1566 semantic similarity. Cross-encoder reranks top-8. ANALYZE: LLM maps to T1566.001 + T1059.001. Severity=HIGH. requires_more_context=False. CONFIDENCE: +0.10 (2 technique IDs). Score=0.82 → band=HIGH. ROUTE: No CRITICAL override (T1059/T1566 not in CRITICAL list). Decision=HIGH. human_review=True (HIGH severity). AUDIT: JSONL record + LangSmith run_id. Total: ~20s.

Q21. What's the conditional edge logic — show the routing conditions.

```
def _should_reretrieve(state): if state.get('error'): return 'confidence' # error propagation. analysis = state.get('analysis', {}); attempts = state.get('retrieval_attempts', 0). if analysis.get('requires_more_context') and attempts < MAX_RETRIEVAL_ATTEMPTS: return 'retrieve' # agentic loop. return 'confidence' # proceed. Registered as: g.add_conditional_edges('analyze', _should_reretrieve, {'retrieve': 'retrieve', 'confidence': 'confidence'}).
```

Q22. How does your human review node interface with an actual analyst workflow?

[HONEST] In the current implementation, 'human review' means the graph pauses and the FastAPI endpoint exposes /human-review/override accepting a JSON payload. In a real SOC, this integrates with the analyst's workstation: the alert appears in the SOC dashboard (ServiceNow, Jira, or custom SIEM UI) with the LLM's analysis pre-populated. The analyst clicks Approve/Override and the selection calls the API. The dashboard integration is [FUTURE]. The LangGraph checkpoint mechanism is fully built and tested.

Q23. How do you handle LLM JSON parse failures?

3-pass recovery: (1) direct json.loads(); (2) strip markdown code fences with regex; (3) regex search for outermost {...} block. If all three fail, node retries up to MAX_RETRIES times with a correction message. If still failing, returns NEEDS_REVIEW error state. The pipeline always produces a decision — even if that decision is 'I failed to parse this.' Identical to LendFlow's extract node — a proven pattern for local models.

Q24. How does error handling work across nodes?

Every node checks `state.get('error')` at entry and returns `{}` immediately if set. Errors accumulated with 'error' and 'error_node' fields. Graph always reaches audit node regardless of errors — complete audit record even for failed runs. A failed triage is still a triage event that needs to appear in the audit trail.

Q25. What's the test strategy?

test_intake.py: source detection, metadata extraction, CVE/IP regex — no LLM. test_routing.py: severity mapping, technique overrides, confidence band logic — no LLM. test_e2e.py: end-to-end with mocked OpenAI client — full graph traversal, state fields, token log accumulation, audit ID generation. GitHub Actions runs intake + routing unit tests on every PR + mocked E2E. eval.yml runs RAGAS evaluation separately.

Q26. Why FastAPI instead of Flask?

Async-native, Pydantic request/response validation built-in, automatic OpenAPI docs at /docs, type-annotated routes. FastAPI + LangGraph's astream() gives a natural async story. Pydantic validation means malformed alert payloads are rejected at the API layer before reaching the graph. Standard in 2025.

Q27. How would you scale this to 10,000 alerts per day?

10,000/day = 417/hr. Replace LM Studio with OpenAI API (3 env vars). Kafka for alert ingestion — workers call graph.astream() async. PostgresSaver for HITL state. Redis cluster for cache. Qdrant for retrieval. S3 + DynamoDB for audit. With 2 async workers and GPT-4o-mini at ~3s/alert: capacity = $2 \times 3600/3 = 2,400/\text{hr}$ — well above 417. Zero pipeline code changes required.

Q28. How does this compare to LendFlow in skill demonstration?

LendFlow: document pipeline, PII redaction, structured extraction, deterministic rule engine, hybrid RAG for policy retrieval, HITL, audit logging. ARL: agentic feedback loop (different problem class), observability-first design with LangSmith, RAGAS retrieval evaluation, security domain (MITRE/NIST/CISA), deterministic technique overrides. Together they cover Applied LLM Systems + AI Infrastructure across two completely different verticals.

Q29. If you had to pick one thing that would impress a senior engineer?

The two-graph compilation pattern. Compiling with interrupt_before=['audit'] means the audit node never runs in batch/demo/eval contexts. Discovering that, understanding why (LangGraph's interrupt semantics), and fixing it cleanly by exposing two named graph variants — not a flag or try/except — shows you've actually run this in production conditions.

Q30. Explain PSI and why you'd use it for alert volume monitoring.

$\text{PSI} = \sum (P_{\text{actual}} - P_{\text{expected}}) \times \ln(P_{\text{actual}} / P_{\text{expected}})$ over decile bins. For alert triage, PSI on severity distribution catches: alert storm (sudden HIGH spike), false positive wave (BENIGN rate collapses), or model degradation (MEDIUM inflates). Thresholds: <0.10 stable, 0.10-0.20 moderate, >0.20 investigate. Same PSI implementation used in RiskFrame for credit score drift monitoring.

Q31. How would you implement alert deduplication at scale?

Current cache key: SHA-256(alert_text + retrieval_attempt). Production: extend to SHA-256(normalized_alert + source_system + time_bucket) where time_bucket is a 5-minute window. Store in Redis with SETNX (atomic set-if-not-exists) — if SETNX returns 0, alert already processed in this window, return cached result. Edge case: same text from two different source IPs should be treated as distinct — include source_system in the hash key.

Q32. How would you evaluate whether the LLM is hallucinating technique IDs?

Three checks: (1) RAGAS faithfulness — if LLM outputs T1234 but it's not in retrieved_chunks, faithfulness drops. (2) Technique ID validation — post-analysis, validate each ID against the MITRE corpus index; unknown IDs are flagged. (3) Deterministic override consistency — if LLM outputs T1486 but doesn't route to CRITICAL, the override catches the inconsistency. Three-layer check is more robust than a single metric.

Q33. What's your approach to prompt engineering for the analyze node?

Structured prompt: (1) system role as security analyst, (2) alert text, (3) retrieved chunks with source labels (MITRE/NIST/CISA), (4) explicit JSON output schema (TriageAnalysis fields), (5) instruction to set requires_more_context=true only if specific technique context is missing. Key: output schema embedded in prompt, not just described — model sees the exact JSON structure. Reduces parse failures on smaller models. Prompt is versioned in config.

Q34. How does token cost scale with re-retrieval?

First pass: ~500 (system) + ~300 (alert) + ~3200 (8 chunks × 400 chars) = ~4000 tokens. Second pass: same + ~200 tokens partial analysis context = ~4400 tokens. Total 2-pass alert: ~8400 tokens. GPT-4o-mini at \$0.15/1M input: ~\$0.00126/alert, \$1.26/1000 alerts. Token economics logged in token_log — visible in LangSmith.

Q35. Describe the Pydantic TriageAnalysis model and why it matters.

Fields: technique_ids: List[str], tactics: List[str], severity: Literal['CRITICAL','HIGH','MEDIUM','LOW','BENIGN','UNKNOWN'], confidence: float, requires_more_context: bool, recommended_controls: List[str], iocs: List[str], severity_rationale: str. Why: (1) validates LLM output structure before propagating to downstream nodes, (2) type safety in AlertState TypedDict, (3) clean serialization to audit log. Without Pydantic, a missing severity field causes KeyError in route node.

Q36. What security considerations apply to the audit log itself?

No raw alert text (only alert_id), no chunk text (only chunk IDs), no PII, no IOCs (only ioc_count). Append-only JSONL — no UPDATE or DELETE. The LangSmith run_id links to the trace but LangSmith is a separate system with its own access controls. Production: audit logs encrypted at rest, access-controlled to SOC leadership, retained per CERT-In's 180-day incident record requirement.

Q37. How would you A/B test two retrieval strategies in production?

Route alerts to strategy A vs B based on alert_id hash modulo 2 (deterministic split). Both strategies write retrieved_chunks and scores to AlertState. RAGAS evaluation on both sets post-hoc. Analyst override rate is the proxy for retrieval quality — more overrides means worse retrieval. Key constraint: both strategies produce identical audit log schema. LangSmith traces separate by strategy tag.

Q38. What is the audit log's schema designed to prevent?

Three things: (1) PII leakage — no raw alert text, no IOC values, only ioc_count and chunk IDs; (2) storage inflation — chunk IDs not chunk text ($400 \text{ chars} \times 8 \times 10\text{k alerts/day} = 32\text{MB/day}$ avoided), same pattern as LendFlow's pii_map_id; (3) tampering — append-only JSONL with SHA-256 alert hash enables deduplication and tamper detection. At 10k alerts/day, audit log grows at ~5MB/day without chunk text.

Q39. How does the pipeline version field in AlertState help in production?

pipeline_version is stored in every audit record. When you upgrade the LLM, change retrieval strategy, or update the prompt, you bump the version. This means you can: (1) filter audit records by version to compare decision distributions before/after deployment, (2) identify which alerts were processed under a buggy version for retroactive review, (3) match LangSmith traces to code versions for incident investigation. Without this, debugging production regressions is extremely hard.

Q40. What would you add to make this production-ready?

1. Full MITRE ATT&CK; STIX/TAXII feed ingestion (not curated excerpts). 2. Real-time CISA KEV feed polling via scheduler. 3. Async alert ingestion via Kafka topic. 4. Structured LLM output via function calling (eliminates JSON recovery logic). 5. Isotonic calibration of confidence thresholds on labelled alert history. 6. Session-state multi-stage attack context tracking per source entity. 7. CrowdStrike/Splunk/SentinelOne ingestion adapters. 8. Rate limiting, auth, tenant isolation on FastAPI layer.

Q41. What are the RAGAS metric values and how do you interpret each?

context_recall 0.923: retrieval fetches the right chunks ~92% of the time — BM25 captures exact IDs, dense captures paraphrases, together they complement. answer_relevancy 0.891: triage output is specific to the alert, not generic security boilerplate. faithfulness 0.847: analysis is mostly grounded in retrieved chunks; the 15% gap indicates some off-chunk model priors, especially for techniques not well-represented in the corpus. context_precision 0.814: cross-encoder reranker earns its place — without it precision was 0.791.

Q42. How does your project address the LLM reliability problem in safety-critical contexts?

Four layers: (1) deterministic technique hard overrides — CRITICAL path decisions never depend on LLM probability; (2) bounded agentic loop — MAX_RETRIEVAL_ATTEMPTS=2 enforced by state counter, not LLM self-termination; (3) HITL interrupt — CRITICAL/HIGH always surface to human analyst before audit record is written; (4) error propagation — every node returns a valid (possibly error-state) AlertState so audit always runs. The system degrades gracefully — it never silently drops an alert.

Defense Pack Summary

- Total Q&A; pairs: 42 (expanded from original 26)
- All claims labelled: [IMPLEMENTED] / [SIMULATED] / [PARTIALLY VERIFIED] / [FUTURE]
- Failure modes taxonomy: 9 failure modes with triggers, mitigations, and status
- Scalability section: 9 bottlenecks with production fixes and target metrics
- Resume claim audit: 11 claims checked against code evidence
- Do Not Say section: 7 overstatement patterns with honest alternatives
- Cross-project links: LendFlow (same BM25+dense stack), RiskFrame (same PSI concept), MetricLens (same audit trail pattern)